# 학습 노트: 다국어 라우터 패턴 + PR diff 파싱

작성일: 2026-05-24 관련 파일: `dev2/tools/` (python/javascript/csharp/html/css/common), `dev2/nodes/` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 3 — CLI 도구 + 다국어 라우터 구현

---

## 한 줄 요약

PR에 들어온 파일의 확장자(.py, .js, .cs 등)를 보고 "이 파일은 어떤 검사 도구로 보낼지" 자동으로 분류·전달하는 설계. 우체국이 우편물의 주소를 보고 지역별 트럭에 나눠 싣는 것과 같다.

---

## 개념 1. 다국어 라우터 패턴

**(1)** 비유

우체국에 들어온 택배 더미(PR 변경 파일들)를 분류원이 주소(확장자)를 보고 - 서울행 → 서울 트럭 (.py → Python 도구) - 부산행 → 부산 트럭 (.js → JavaScript 도구) - 제주행 → 제주배 (.cs → C# 도구) 로 나눠 싣습니다. 분류원은 택배 안에 뭐가 들었는지 몰라도, 주소만 보고 정확한 트럭으로 보냅니다.

**(2)** 명확한 개념

> 다국어 라우터(Router)는 입력(파일)을 일정한 규칙(확장자 → 언어)에 따라 적절한 처리기(언어별 도구 모음)로 분배하는 디스패치**(dispatch)** 패턴이다. 핵심은 "분배 로직"과 "실제 처리 로직"을 분리하는 것 — 라우터는 어디로 보낼지만 알고, 검사 방법은 각 언어 모듈이 책임진다.

핵심 원리: 개방-폐쇄 원칙**(OCP)**. 새 언어(예: Go)를 추가할 때 라우터 본체는 거의 건드리지 않고, 매핑 테이블에 한 줄과 새 도구 모듈만 추가하면 된다.

**(3)** 왜 쓰나

- 없으면: 한 함수 안에 `if .py: pylint... elif .js: eslint... elif .cs: ...` 이런 if-else가 끝없이 늘어남 → 언어 추가할 때마다 거대한 함수 수정 → 버그 위험 + 한 사람만 건드릴 수 있는 병목
- 있으면: 확장자→도구 매핑 테이블 한 곳만 관리 → 언어별 폴더가 독립적이라 팀원이 각자 다른 언어 도구를 동시에 개발 (CLAUDE.md §15 병렬 개발과 맞물림)

**(4)** 실제 코드

```
# dev2/tools/common/router.py
from dev2.tools.python import run_python_tools
from dev2.tools.javascript import run_javascript_tools
```

```python
from dev2.tools.csharp import run_csharp_tools
from dev2.tools.html import run_html_tools
from dev2.tools.css import run_css_tools

# 확장자 → 언어 매핑 테이블 (새 언어는 여기에 한 줄 추가)
EXT_TO_LANG = {
    ".py": "python",
    ".js": "javascript", ".jsx": "javascript",
    ".ts": "javascript", ".tsx": "javascript",
    ".cs": "csharp",
    ".html": "html",
    ".css": "css",
}

# 언어 → 도구 실행기 매핑 (라우터의 핵심: dispatch 테이블)
LANG_TO_RUNNER = {
    "python": run_python_tools,
    "javascript": run_javascript_tools,
    "csharp": run_csharp_tools,
    "html": run_html_tools,
    "css": run_css_tools,
}

def route_files(files: list[str]) -> dict[str, list[str]]:
    """파일 목록을 언어별로 그룹핑. 예: {'python': ['a.py'], 'javascript':
['b.js']}"""
    grouped: dict[str, list[str]] = {}
    for f in files:
        ext = f[f.rfind("."):].lower() if "." in f else ""
        lang = EXT_TO_LANG.get(ext)
        if lang is None:
            continue   # 지원 안 하는 확장자(.md, .txt 등)는 건너뜀
        grouped.setdefault(lang, []).append(f)
    return grouped
```

```python
# dev2/nodes/run_tools.py — 라우터 + 비동기 동시 실행 통합
import asyncio

async def run_all_tools(files: list[str]) -> list[dict]:
    grouped = route_files(files)   # 언어별로 묶고
    # 언어별 도구 실행기를 동시에 실행 (asyncio.gather)
    tasks = [LANG_TO_RUNNER[lang](langfiles) for lang, langfiles in
grouped.items()]
    results = await asyncio.gather(*tasks, return_exceptions=True)

    issues: list[dict] = []
    for r in results:
        if isinstance(r, Exception):
            continue   # 특정 언어 도구가 실패해도 나머지는 살림
```

```
        issues.extend(r)
    return issues
```

---

## 개념 2. PR diff 파싱 (unified diff → 변경 파일 목록)

**(1)** 비유

편집자가 빨간 펜으로 고친 원고를 받았을 때, "어떤 페이지를(파일) 몇 번째 줄을(line) 고쳤는지" 표시(+++, @@)만 보고 빠르게 찾아내는 것. 원고 전체를 다시 읽지 않고 표시만 따라갑니다.

**(2)** 명확한 개념

> GitHub Webhook이 보내는 `pr_diff`는 **unified diff** 형식의 텍스트다. 여기서 `+++ b/`경로 줄은 "변경된 파일 경로", `@@ -a,b +c,d @@` 줄(hunk 헤더)은 "몇 번째 줄이 바뀌었는지"를 알려준다. 라우터는 먼저 이 diff에서 변경된 파일 목록을 뽑아내야 어떤 언어 도구로 보낼지 결정할 수 있다.

unified diff 구조 읽는 법:

```
diff --git a/src/auth.py b/src/auth.py   ← 파일 시작
--- a/src/auth.py                          ← 변경 전 파일
+++ b/src/auth.py                          ← 변경 후 파일 (★ 여기서 파일 경로 추출)
@@ -10,3 +10,5 @@ def login():            ← hunk 헤더: +10,5 = 10번째 줄부터 5줄
-    query = "SELECT * FROM users"          ← 삭제된 줄 (앞에 -)
+    query = f"SELECT * FROM users WHERE id={uid}"  ← 추가된 줄 (앞에 +)
```

**(3)** 왜 쓰나

- 없으면: PR 전체 코드를 다 검사 → 안 바뀐 수백 개 파일까지 검사해 느리고, "이번 PR이 만든 문제"와 "원래 있던 문제"를 구분 못 함
- 있으면: 바뀐 파일·줄만 추려 검사 → 빠르고, 리뷰 코멘트를 정확히 바뀐 줄(line)에 달 수 있음 (Output Contract의 `line` 필드)

**(4)** 실제 코드

```python
# dev2/tools/common/diff_parser.py
import re

def parse_changed_files(pr_diff: str) -> list[str]:
    """unified diff 에서 변경된 파일 경로 목록 추출."""
    files: list[str] = []
    for line in pr_diff.splitlines():
        # "+++ b/src/auth.py" 형태에서 경로만 뽑음
        if line.startswith("+++ b/"):
            path = line[len("+++ b/"):].strip()
```

```
            if path != "/dev/null":   # 삭제된 파일은 제외
                files.append(path)
    return files


def parse_changed_lines(pr_diff: str) -> dict[str, set[int]]:
    """파일별로 '추가/변경된 줄 번호' 집합 추출 (리뷰 코멘트 위치용)."""
    result: dict[str, set[int]] = {}
    current_file = None
    new_line_no = 0
    for line in pr_diff.splitlines():
        if line.startswith("+++ b/"):
            current_file = line[len("+++ b/"):].strip()
            result.setdefault(current_file, set())
        elif line.startswith("@@"):
            # "@@ -10,3 +10,5 @@" 에서 +10 (변경 후 시작 줄) 추출
            m = re.search(r"\+(\d+)", line)
            new_line_no = int(m.group(1)) if m else 0
        elif line.startswith("+") and not line.startswith("+++"):
            if current_file:
                result[current_file].add(new_line_no)   # 추가된 줄 기록
            new_line_no += 1
        elif not line.startswith("-"):
            new_line_no += 1   # 변경 없는 줄도 줄 번호는 증가
    return result
```

---

## 두 개념이 합쳐지는 전체 흐름 (단계별)

1. **Dev1**이 **PR diff** 전달: `PRReviewState.pr_diff`에 unified diff 텍스트가 담겨 옴
2. **diff** 파싱: `parse_changed_files()`로 변경 파일 목록 추출 → `["src/auth.py", "web/app.js"]`
3. 라우팅: `route_files()`로 언어별 그룹핑 → `{"python": ["src/auth.py"], "javascript": ["web/app.js"]}`
4. 비동기 실행: 언어별 도구 실행기를 `asyncio.gather`로 동시 실행 (subprocess로 pylint·eslint 등)
5. 결과 통합 + 위치 매핑: 각 이슈를 `parse_changed_lines()` 결과와 대조해 "이번 PR이 바꾼 줄의 이슈"만 추려 `line`에 정확히 매핑
6. **Output Contract** 변환: `schema.py`의 `AgentOutput`으로 묶어 Dev4에게 전달

---

## 3rd-Project-sub2 프로젝트에서

파일: `dev2/tools/common/router.py`(라우터), `dev2/tools/common/diff_parser.py`(diff 파싱), 언어별 `dev2/tools/python·javascript·csharp·html·css/`

역할: - `dev2/tools/` 폴더가 이미 언어별로 나뉘어 있는 이유가 바로 이 라우터 패턴 때문 — 각 폴더가 독립적이라 팀원이 언어별로 병렬 개발 가능 (CLAUDE.md §15) - 라우터는 "분배만" 담당하고 실제 검사는 각 언어 모듈이 책임 → Clean Architecture(CLAUDE.md §2)의 관심사 분리와 일치 - diff 파싱으로 변경 줄만 추려야 Output Contract의 `line` 필드가 정확해지고, Dev4가 GitHub에 정확한 위치로 코멘트를 달 수 있음

---

## 연관 개념

- **subprocess / asyncio** → 라우팅 후 각 언어 도구를 비동기 실행 (`docs/learning/ai/subprocess_async.md`)
- 정적 분석 도구 → 라우터가 보내는 목적지(`docs/learning/ai/static_analysis_tools.md`)
- 디스패치 테이블**(dispatch table)** → if-else 대신 딕셔너리로 분기를 관리하는 기법. 라우터의 핵심
- 개방-폐쇄 원칙**(OCP)** → 기존 코드 수정 없이 새 언어를 추가할 수 있는 설계 원칙
- **unified diff** → Git이 변경 내용을 표현하는 표준 텍스트 형식 (`+++`, `@@` 마커)

---

## 복습 퀴즈 (3문항)

1. **(이해)** 다국어 라우터에서 "분배 로직"과 "검사 로직"을 분리하면 좋은 점은? Go 언어를 새로 추가하는 상황으로 설명하라.
   - 답: 라우터(분배)는 어디로 보낼지만 알고 검사 방법은 각 언어 모듈이 책임지므로, Go 추가 시 라우터 본체는 거의 안 건드리고 `EXT_TO_LANG`에 `".go"` 한 줄, `LANG_TO_RUNNER`에 매핑 한 줄, 그리고 `tools/go/` 모듈만 추가하면 된다(개방-폐쇄 원칙). 거대한 if-else를 수정할 필요가 없어 버그 위험과 병목이 줄어든다.
2. **(적용)** unified diff에서 변경된 "파일 경로"는 어떤 줄에서, 변경된 "줄 번호"는 어떤 줄에서 얻는가?
   - 답: 파일 경로는 `+++ b/`경로 줄에서 얻고(`/dev/null`은 삭제 파일이라 제외), 변경 줄 번호는 `@@ -a,b +c,d @@` hunk 헤더의 `+c`(변경 후 시작 줄)부터 시작해 `+`로 시작하는 추가 줄을 세어 계산한다.
3. **(비교)** PR 전체 코드를 다 검사하는 것과, diff로 변경 파일/줄만 검사하는 것의 차이 두 가지는?
   - 답: (1) 속도 — 안 바뀐 수백 개 파일까지 검사하면 느리지만, 변경분만 검사하면 빠르다. (2) 정확성 — 변경 줄만 보면 "이번 PR이 만든 문제"와 "원래 있던 문제"를 구분할 수 있고, Output Contract의 `line` 필드로 GitHub의 정확한 위치에 리뷰 코멘트를 달 수 있다.